

Data Structures

generic



2017/2018

Generic Methods

- This method is intended to print the contents of a *array*:

```
public static void print (string [] m)
{
    for (String a: m)
        System.out.println (a);
}
```

```
public static void main (String args []) {
    String m[] = { "Joe", "Anthony"};
    print (m);
}
```



Generic Methods

- The previous function only works for *arrays* of *strings*.
- However, it may be *generalized* to accept *arrays* with any kind of elements

```
public static <T> void print (T [] m)
{
    for (T a: m)
        System.out.println (a);
}
```

```
public static void main (String args []) {
    String m[] = { "Joe", "Anthony"};
    Integer n [] = {2, 3};
    print (m);
    print (n);
}
```



Generic Methods

```
public static <T> T chooseRandom (T a, T b)
{
    T ret;
    if (Math.random () > 0.5)
        ret = a;
    else
        ret = b;
    return ret;
}
```



Generic classes

- The generic concept can also be applied to classes.

```
public class Pair <T, S> {  
    T first;  
    S second;  
    public Pair (T a, S b) {  
        first = a;  
        second= b;  
    }  
    public T getFirst() {return first;}  
    public S getSecond() {return second;}  
  
    Public void setFirst (T a){first=a;}  
    public void setSecond (S b) {second = b;}  
}
```



Version without generics

- However, it is also possible to obtain a similar behavior through the use of the class *Object*. So, what is the benefit of using generics?

```
public class Pair {  
    Object first;  
    Object second;  
    public Pair (Object a, Object b) {  
        first = a;  
        second= b;  
    }  
    public Object getFirst() {return first;}  
    public Object getSecond() {return second;}  
  
    public void setFirst (Object a){first=a;}  
    public void setSecond (Object b) {second = b;}  
}
```



Comparison - Version No Generic

```
public static void main (String args [])  
{  
    P1 = new Pair( "Par1", 1);  
    P2 = new Pair(2 "Par2");  
  
    Integer i1 = (Integer) (p1.getSecond ()); //OK  
    Integer i2 = (Integer) (p2.getSecond ());  
        // runtime error! - ClassCastException  
}
```



Comparison - Generic Version

```
public static void main (String args [])  
{  
    // Java 7+  
    Pair <String, Integer> p1 = new Pair <> ( "Par1", 1);  
  
    // Java 5,6  
    Pair <Integer, String> p2 =  
        new Pair <Integer, String> (2 "Par2");  
  
    Integer i=p1.getSecond(); //OK  
    Integer i2=p2.getSecond(); // compile error!  
}
```

The use of generics instead of the class *Object* enables detection of type errors at compile time.



wildcards

- You can impose restrictions on the formal parameters. For example, in this case, T:

```
public static <T extends Number> boolean isLarger(T p1, T p2) {  
    return p1.doubleValue ()> p2.doubleValue ();  
}  
public static void main (String args []) {  
    if (isLarger(33,4))  
        System.out.println ( "bigger");  
    Boolean b = isLarger( "Hello", "Goodbye"); // compilation error  
        // String does not extend Number.  
}
```



wildcards

- *wildcards* can be specified as follows:
 - **X extends Y :**
 - X is Y or extends Y (or X implements interface Y)
 - **X super Y:**
 - Y is equal to or extends class X (or X implements interface Y)
 - Sometimes, X or Y can be replaced by "?", Which means "any class".



wildcards - Example

- The ? *wildcard* can be used to replace formal parameters that are used only once:
- Example 1:
 - `public static <T, S> void f (pair <T, S> w)`could can be written as
 - `public void f (pair <?,?> w)`
- Example 2:
 - `public static <T, S extends T> void f (pair <T, S> w)`
- Could be written as
 - `public static <T> void f (pair <T,? extends T> p)`



Polymorphism support with Wildcards

```
public class Pair <T, S> {  
    T first;  
    S second;  
    public Pair (T a, S b) {  
        first = a;  
        second= b;  
    }  
    public T getFirst() {return first;}  
    public S getSecond() {return second;}  
  
    ...  
  
    public void copy (Pair <T, S> otherPair)  
    {  
        first = otherPair.getFirst();  
        second= otherPair.getSecond();  
    }  
}
```



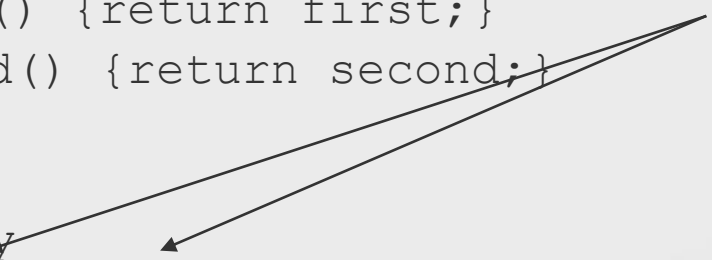
Polymorphism support with Wildcards

```
class Rect {...  
};  
class Quad extends Rect {  
...  
}  
public static void main (String args [])  
{  
    Quad q1 = new Quad (1), q2 = new Quad (2);  
    Rect R1 = new Rect (2,3), r2 = new Rect (3,4);  
    Pair <Quad, Rect> p1 = new Pair <> (q1, r1);  
    Pair <Quad, Rect> p2 = new Pair <> (q2, r2);  
    p2.copy (p1); //OK  
    Pair <Rect, Rect> p3 = new Pair <> (q1, r1);  
    Pair <Rect, Rect> p4 = new Pair <> (q2, r2);  
    p3.copy (p4); //OK  
    p3.copy (p1); // compilation error:  
    // Pair <Quad, Rect> can not be used  
    // where Pair <Rect, Rect> is required  
}
```



Polymorphism support with Wildcards

```
public class Pair <T, S> {  
    T first;  
    S second;  
    public Pair (T a, S b) {  
        first = a;  
        second= b;  
    }  
    public T getFirst() {return first;}  
    public S getSecond() {return second;}  
  
    ...  
    public void copy  
        (Pair <? extends T ,? extends S> otherPair)  
        {  
            first = otherPair.getFirst();  
            second= otherPair.getSecond();  
        }  
}
```



Polymorphism support with Wildcards

```
class Rect {...  
};  
class Quad extends Rect {  
...  
}  
public static void main (String args [])  
{  
    Quad q1 = new Quad (1), q2 = new Quad (2);  
    Rect R1 = new Rect (2,3), r2 = new Rect (3,4);  
    Pair <Quad, Rect> p1 = new Pair <> (q1, r1);  
    Pair <Quad, Rect> p2 = new Pair <> (q2, r2);  
    p2.copy (p1); //OK  
    Pair <Rect, Rect> p3 = new Pair <> (q1, r1);  
    Pair <Rect, Rect> p4 = new Pair <> (q2, r2);  
    p3.copy (p4); //OK  
    p3.copy (p1); // OK!  
}
```



Polymorphism and Wildcards

Consider the following hierarchy:

```
class Figure implements Comparable <Figure> {  
    int compareTo (Figure F) {...}  
}
```

```
class Rect extends Figure {...}
```

You can do this:

```
Rect r = new Rect(2,3), q = new Rect (3,4);  
r.compareTo(q);  
// is valid, although Rect is not Comparable <Rect>  
// because it is Comparable <Figure>  
// why Comparable <Figure>
```

So, how to use *wildcards* to define a method that takes a *Rect* parameter and any other object that can be compared with him?



Polymorphism and Wildcards

Attempt 1:

```
static int compare (Rect r, Comparable <Rect> r) {  
    // does not work well ... Any figure is comparable  
    // with Rect  
}
```

Attempt 2:

```
static int compare (Rect r, Comparable <Figure> c) {  
    // It works better, but unfortunately requires us to  
    // know and specify exactly the comparable class (Figure)  
  
    // If is also possible to compare another class not  
    // related figure (eg: Color) with a  
    // rectangle, then will not work in this case  
}
```

How to generalize?



Polymorphism and Wildcards

Just make sure the received object parameter is compared with a superclass of *rect*!

```
static int compare (Rect r Comparable <? super Rect> c) {  
    ...  
    // It works well in all cases  
    // remember a Rect object that can be comparable with  
    // anything comparable to a Rect, Figure or even Object  
    // ...  
}
```

generalizing the *rect* so that the function can receive any kind of objects

```
static <T> int compare (T r, Comparable <? super T> c) {  
    return c.compareTo (r); //OK!  
}
```



Limitations of Generics

- When a program is compiled, the parameters of generics are removed, being used *Objects* (Or other suitable superclass) in place.
 - *The generic information is used only during the build process to do type checking.*
 - *This process is called Type Erasure*
- This conditions the possible operations with these types.

It is implemented as ...
→

```
public class value <T> {  
    T obj;  
    T f(T x) {...}  
}
```

```
public class Value {  
    Object obj;  
    Object f(Object x) {...}  
}
```



Limitations of Generics

- T being a formal parameter, you can not:
 - Create arrays of parameterized classes:
 - `Pair <X, Y> m [] = new Pair <> [10];`
 - T type Object creation:
 - `T obj = new T ();`
 - Extension:
 - `class <X> extends X`
 - Using the type in static members or methods:
 - `private static T value;`
 - `public static void f (T in);`
 - dynamically obtain information on the type:
 - `x instanceof T`
 - using in enumerations:
 - `enum State <X> {...}`



Exercise

- Define a ComparablePair class, knowing that this is a Pair whose elements are comparable. The ComparablePair class must in turn be also comparable: the comparison must be determined by comparing the first two elements of each pair, resolving ties by comparing second elements.

